AgentOps 项目面试题库与追问树

副标题：面向 AI 应用开发 / Agent 开发岗位
版本：2026-03 AgentOps 2.2 当前仓库对齐版
适用场景：项目深挖、简历追问、开场讲稿、连续高压追问、双项目区分

使用说明

这份文档不再沿用旧版题库叙事，而是以当前仓库实现为唯一真相，专门服务 `AI 应用开发 / Agent 开发` 岗位。

- 回答顺序固定成：`项目定位 -> 三层架构 -> LangGraph runtime -> checkpoint / replay / approval -> tool plane / MCP -> event stream / trace -> runtime-based Eval -> 诚实边界 -> 与 AISmartQA 区分`
- 面试时先讲系统价值，再讲框架名词，不要一上来把项目讲成“我用了 LangGraph、LangChain、Gemini”
- 这份文档只讲当前实现，不单独展开旧版升级故事
- 如果某项能力仓库里没有，就明确说“当前边界如此，后续可以增强”，不要硬说已经上线

当前事实来源

这份文档默认以下材料优先级最高：

- `AgentOps/README.md`
- `AgentOps/docs/01-Demo演示路径.md`
- `AgentOps/docs/03-简历项目描述与讲稿.md`
- `AgentOps/docs/05-10分钟稳定讲稿与追问树（强化版）.md`
- `AgentOps/docs/06-JD映射矩阵.md`
- `AgentOps/agentops-app`、`AgentOps/agentops-runtime-py`、`AgentOps/agentops-web` 中的当前实现

一句话定位

`AgentOps` 是一个面向企业知识与数据 Copilot 场景的可治理 Agent 系统，采用 `Java control plane + Python LangGraph runtime + React workspace` 三层结构，把 `multi-step orchestration、checkpoint replay、human approval、tool governance、event stream、runtime-based Eval` 真正落成了一条可运行、可追踪、可恢复、可评测的主链。

高频首问

1. 这个项目到底解决什么问题？
2. 为什么它不是普通 RAG 或 workflow demo？
3. 为什么要拆成 `Java control plane + Python runtime + React workspace`？
4. `LangChain` 和 `LangGraph` 在项目里分别落在哪一层？
5. 为什么这个项目更像 AI 应用 / Agent 岗会问的系统？

强背清单

- `AgentOps` 不是“LangGraph demo”，而是 `Java control plane + Python runtime + React workspace` 的 Agent 系统
- Java 统一持有 `session / run / step / event / human_task / checkpoint / eval`，是系统记录源
- Python runtime 用 LangGraph 承接真实图执行，核心节点是 `supervisor_plan / evidence_reviewer / finalize`
- 当前已支持 `SINGLE_GRAPH / TEAM_GRAPH`，后者是受控 multi-role / team-graph，不是开放式 swarm
- checkpoint 已落到 Java / MySQL，非审批态支持从最新 checkpoint 做 `replay / continue`
- 审批恢复仍通过 `resumeToken + checkpointVersion`
- 前端已经支持 `agent_run_event + SSE live event stream`，轮询只做兼容兜底
- Eval 跑的是真实 runtime 图，不是旁路脚本

讲稿锚点

30 秒版本

我做的 `AgentOps` 不是普通 RAG，也不是单轮 `prompt -> answer` 后端，而是一个可治理 Agent 系统。Java 是 control plane，负责鉴权、session、run/step/event、checkpoint、approval、Eval 和 MCP tool plane；Python 用 FastAPI + LangGraph 跑真正的 runtime，核心节点 `supervisor_plan / evidence_reviewer / finalize` 由 Gemini 驱动；前端再把 graph、trace、审批、SSE live events 和 Eval dashboard 展示出来。这个项目的价值不在“调模型”，而在于把运行时、恢复、审批、事件流和评测闭环都做进了系统主链。

1 分钟版本

`AgentOps` 解决的不是单纯问答，而是把企业知识检索、数据查询、人工审批和多步决策收进一个可治理 Agent 系统。当前项目采用三层结构：Java control plane 统一持有 session、run/step、human task、checkpoint、event、Eval 和 MCP；Python runtime 用 LangGraph 跑 `intake / memory / supervisor / tool / review / approval / finalize` 状态图，并支持 `SINGLE_GRAPH / TEAM_GRAPH`、checkpoint replay、skill trace 和 tool governance；React workspace 负责把 graph trace、live event stream、审批和 dashboard 展示出来。这样项目就从“多工具问答”升级成了一个有运行时、有恢复能力、有审批分支、有事件流、有评测闭环的 Agent 系统。

2 分钟稳定讲稿

如果用两分钟介绍，我会先把它定性成一个可治理 Agent 系统，而不是普通 LLM 应用。业务上，它面向企业知识与数据 Copilot 场景，不只是回答一个问题，而是要在知识检索、结构化数据查询、证据审查、人工审批和最终回答之间做多步决策。

架构上我把系统拆成了三层。第一层是 Java control plane，负责鉴权、session/message、run/step/event、human task、checkpoint、memory fact、Eval 和 MCP tool plane。第二层是 Python runtime，用 FastAPI + LangGraph 承接真实图执行，主链是 `intake_guardrail -> load_memory -> supervisor_plan -> knowledge_researcher / data_analyst -> evidence_reviewer -> human_approval -> finalize`，其中 `supervisor_plan / evidence_reviewer / finalize` 由 Gemini 做结构化决策。第三层是 React workspace，负责把聊天记录、graph 视图、step trace、SSE live events、审批面板和 Eval dashboard 展示出来。

当前项目最能体现 Agent 岗价值的点有五个。第一，checkpoint 已经落到 Java / MySQL control plane，不再依赖 Python 进程内状态。第二，恢复不只停留在审批态，非审批态也能从最新 checkpoint 做 replay / continue，而审批恢复仍通过 `resumeToken + checkpointVersion` 做并发校验。第三，工具没有散落在

runtime 里，而是统一经由 Java MCP tool plane 暴露，并带 `riskLevel / approvalPolicy / timeoutBudget / retryPolicy / auditRequired` 这些治理元数据。第四，前端已经支持 `agent_run_event + SSE` 的实时事件流，轮询只是兼容兜底。第五，Eval 跑的是真实 runtime 图，能记录 `route / grounding / citation / approval / latency / retry / orchestrationMode / skills`，所以评测和线上行为是一致的。

因此我现在讲 `AgentOps`，不会强调“我会用 LangGraph”，而是强调“我把运行时、系统记录源、恢复、审批、事件流和评测闭环拆清楚了”。

3 分钟版本提纲

1. 项目解决的是企业知识与数据 Copilot 的多步执行与治理问题，不是单纯问答
2. 三层结构很明确：Java 负责记录源和控制面，Python 负责图执行和决策，React 负责工作台和可视化
3. LangGraph 主链是真实 runtime，不是示意图，`SINGLE_GRAPH / TEAM_GRAPH` 都已经进主链
4. checkpoint、replay、approval 和 `resumeToken + checkpointVersion` 让系统具备恢复与审计能力
5. `agent_run_event + SSE` 让事件流可视化，trace 不再只靠轮询
6. Eval 直接跑真实 runtime 图，把 route、grounding、citation、approval、latency、skills 串成质量闭环
7. 当前边界清晰：不是官方 durable checkpointer，不是完整 SaaS 权限平台，也不是开放式 swarm 平台

第一部分：项目题库

模块一：项目定位与岗位映射

面试题 1：这个项目解决的到底是什么问题？

标准回答

它解决的不是单纯问答，而是把企业知识检索、结构化数据查询、证据审查、人工审批和多步决策收进一个可治理 Agent 系统。普通 LLM 应用经常停留在单轮生成，但企业场景更关心的是多步执行、状态持久化、恢复、风控和评测。

项目落地

`AgentOps` 的主链是：登录 -> 新建会话 -> 创建 Agent Run（`SINGLE_GRAPH / TEAM_GRAPH`）-> Python runtime 拉上下文 -> 走 `intake / memory / supervisor / tool / review / approval / finalize` -> Java 回写 run/step/graph/human-task/checkpoint/eval/events -> Web 通过 SSE 看 event stream。

易错表达

- 易错：把项目讲成“RAG + tool calling”
- 更稳：先讲“多步执行 + checkpoint + approval + eval + event stream”这条系统主链

大厂式继续追问

如果面试官说“这不就是带工具的 Copilot 吗”，你怎么把话题拉回系统治理能力？

面试题 2：这个项目为什么不是普通 RAG 或 workflow demo？

标准回答

普通 RAG 的核心是 `检索 -> 生成`，workflow demo 更多是在展示固定流程。`AgentOps` 多了明确的 runtime 层和 control plane：它会 route、按图执行工具、review 证据、必要时进入人工审批，并把 run/step/checkpoint/event 全部落成可追踪的运行态对象，所以它已经不是一次性问答，也不只是画一条流程线。

项目落地

项目里已经有 `agent_run`、`agent_run_step`、`agent_runtime_checkpoint`、`agent_human_task`、`agent_run_event`、runtime Eval、graph trace 和 approval inbox，这些都超出了普通 RAG demo 的范围。

易错表达

- 易错：只强调“工具比较多”“前端图比较炫”
- 更稳：强调“运行态对象完整、恢复路径明确、审批是正式分支、Eval 跑真实主链”

大厂式继续追问

如果面试官继续说“那你和 workflow engine 的区别是什么”，你会怎么答？

面试题 3：为什么这个项目更能对标 AI 应用 / Agent 开发岗位？

标准回答

因为这类岗位更看重的不是“会不会调一个模型”，而是你有没有做过 `runtime`、`checkpoint`、`approval`、`tool governance`、`event stream`、`eval`、`observability` 这些工程层能力。`AgentOps` 当前的主卖点正是这些能力被组织成了一条完整的 Agent 主链。

项目落地

当前项目已经具备 `Java control plane + Python LangGraph runtime + React workspace`、`SINGLE_GRAPH / TEAM_GRAPH`、checkpoint replay、human approval、SSE event stream、skill trace、runtime-based Eval 和工具治理元数据。

易错表达

- 易错：把岗位对标理解成“会几个热门框架名词”
- 更稳：把 JD 关键词落回 `control plane / runtime / replay / approval / eval` 这些系统能力

大厂式继续追问

如果面试官说“这还不是完整生产平台”，你怎么回答既诚实又不掉价？

模块二：三层架构与系统边界

面试题 4：为什么要拆成 Java control plane 和 Python runtime？

标准回答

因为我想把“系统记录源”和“智能决策层”分开。Java 更适合承接鉴权、session/message、run/step/event、checkpoint、approval、Eval、MCP 这些稳定控制面能力；Python 更适合承接 LangGraph、LangChain tools 和 Gemini 驱动的图执行。这样分层之后，runtime 可以持续演进，但系统记录源不会散。

项目落地

Java 暴露 run、checkpoint、human task、event、Eval 和内部 runtime callback；Python runtime 通过内部 API 拉 session context、回写 step/status/checkpoint/event，并通过 Java MCP 工具面访问 `kb_search` / `doc_fetch` / `sql_query`。

易错表达

- 易错：回答成“因为我 Java 和 Python 都会，所以干脆都用上”
- 更稳：回到“记录源统一、边界清晰、治理能力集中”这三个设计理由

大厂式继续追问

这样不是多了一跳网络吗，为什么值得？

面试题 5：为什么 Python runtime 不直接连数据库？

标准回答

因为那样会把系统记录源拆散。我的目标是让 run、step、checkpoint、approval、event、eval 都由 Java control plane 统一持有，Python 只负责运行时和决策，通过内部 API 与 control plane 交互，通过 MCP 工具访问业务能力。这样更利于审计、权限收口和后续演进。

项目落地

runtime 通过 `/internal/runtime/...` 取上下文、保存 checkpoint、回写状态；工具调用统一通过 Java MCP，而不是让 Python 直接查业务库。

易错表达

- 易错：只说“更解耦”
- 更稳：讲“系统一致视图、审计边界、权限收口”这些更像真实工程设计的收益

大厂式继续追问

如果面试官说“直接让 Python 连库不是更快吗”，你怎么接？

面试题 6：React workspace 在这个项目里为什么不是可有可无？

标准回答

因为这个项目的核心价值之一就是把运行时变成可观察、可演示、可调试的系统。没有工作台，很多能力只能停留在接口层描述；有了 workspace，graph、trace、approval、Eval 和 live events 都能被直接展示出来，项目可信度和演示冲击力会高很多。

项目落地

当前工作台已经支持聊天区、run trace、graph 视图、审批面板、Eval dashboard，以及基于 `agent_run_event` 的 SSE live event stream，轮询只做兼容兜底。

易错表达

- 易错：把前端讲成“只是做个页面方便演示”
- 更稳：强调“这是运行态可观测性和系统可解释性的一部分”

大厂式继续追问

为什么现在能把前端讲成 SSE 实时事件流，而不是再说“主要靠轮询”？

模块三：LangGraph runtime 与 orchestration mode

面试题 7：你现在的 LangGraph runtime 真实在做什么？

标准回答

LangGraph 在这个项目里不是示意图，而是真正承接多步状态化执行。它负责跑 `intake_guardrail -> load_memory -> supervisor_plan -> knowledge_researcher / data_analyst -> evidence_reviewer -> human_approval -> finalize` 这条主链，并维护 route、pending tasks、tool trace、review feedback、approval 状态和 checkpoint 信息。

项目落地

`supervisor_plan / evidence_reviewer / finalize` 都由 Gemini 驱动，runtime 还支持 checkpoint replay、bounded failure、skill trace 和工具治理，说明 LangGraph 已经是主运行时，不是展示层。

易错表达

- 易错：把 LangGraph 讲成“我画了个状态图”
- 更稳：讲“它承接真实节点执行、状态流转和恢复逻辑”

大厂式继续追问

如果 Gemini 没 key、结构化输出失败或者节点异常，runtime 怎么收口？

面试题 8：`LangChain` 和 `LangGraph` 在项目里分别落在哪一层？

标准回答

`LangChain` 更偏模型与工具集成层，`LangGraph` 更偏状态化运行时层。在 `AgentOps` 里，LangChain 主要用来承接 Gemini 模型接入和 `StructuredTool` 风格的工具抽象；LangGraph 负责图执行、状态流转、审批分支、恢复和多步 orchestration。项目价值不在框架名词本身，而在 control plane 和 runtime 之间的系统边界被固定下来了。

项目落地

runtime 中真实存在 `ChatGoogleGenerativeAI`、`StructuredTool`、图执行主链和 Java-backed checkpoint；而 session、run、checkpoint、event、Eval 都仍由 Java control plane 统一持有。

易错表达

- 易错：回答成“LangChain 更高级，LangGraph 更底层”
- 更稳：结合项目说“LangChain 负责模型和工具接入，LangGraph 负责状态化执行”

大厂式继续追问

为什么一个复杂 Agent 系统不能只靠高层框架名词讲清楚？

面试题 9：`SINGLE_GRAPH` 和 `TEAM_GRAPH` 分别解决什么问题？

标准回答

`SINGLE_GRAPH` 适合默认单图执行，链路更直接、成本更低；`TEAM_GRAPH` 适合把复杂问题拆成受控 multi-role / multi-skill 协作，但它不是开放式 swarm，而是有边界、有预算的 team-graph。这样设计的好处是你既能覆盖单图场景，也能回答多角色编排相关的岗位追问。

项目落地

当前 README 和 Demo 文档都已经把 `SINGLE_GRAPH / TEAM_GRAPH` 写进主链，Eval 维度里也保留 `orchestrationMode`，可以直接比较不同编排模式的执行表现。

易错表达

- 易错：把 `TEAM_GRAPH` 讲成“多 agent 自动互相商量”
- 更稳：明确它是“受控 multi-role 编排”，不是开放式 swarm 平台

大厂式继续追问

如果面试官追问“为什么不直接一开始就做开放式 multi-agent”，你怎么回答？

面试题 10：为什么要把 `supervisor_plan`、`evidence_reviewer`、`finalize` 拆成独立节点？

标准回答

因为这三个节点分别回答不同问题：supervisor 决定“下一步该做什么”，reviewer 判断“当前证据够不够、要不要重规划、要不要审批”，finalize 负责“如何基于现有证据生成最终答案或 bounded failure”。拆开之后，trace 更清楚，调参更明确，Eval 也更容易按阶段拆指标。

项目落地

当前 runtime 主链已经把这三个 Gemini 节点作为核心决策点，并围绕它们沉淀 route、review feedback、approval reason、node path 和 retry 等运行态数据。

易错表达

- 易错：回答成“拆开只是为了 prompt 更优雅”
- 更稳：强调“责任边界、trace 清晰度、评测可拆分”这些工程收益

大厂式继续追问

为什么 reviewer 和 supervisor 合并成一个大 prompt 反而更难治理？

模块四：checkpoint / replay / approval / recovery

面试题 11：为什么 checkpoint 一定要落在 Java / MySQL control plane？

标准回答

因为 checkpoint 是运行态事实，不应该只存在 Python 进程内，也不应该漂在非主记录源里。把 checkpoint 放到 Java / MySQL control plane 后，run、step、approval、checkpoint、event 都处在同一套记录视图里，直接带来恢复、审计和并发校验三类收益。

项目落地

当前 Java 侧已经有 `agent_runtime_checkpoint` 和对应内部接口，run 上同步维护 `checkpointVersion` 与 `lastCheckpointAt`，runtime 每个关键节点执行后都会落 checkpoint。

易错表达

- 易错：把 checkpoint 讲成“缓存一份状态，方便恢复”
- 更稳：讲“它是主运行态的一部分，不是性能缓存”

大厂式继续追问

为什么这里不直接用 Redis 或完全依赖官方内存态？

面试题 12：`resumeToken` 和 `checkpointVersion` 分别解决什么问题？

标准回答

`resumeToken` 解决“是不是这次暂停态”的身份问题，`checkpointVersion` 解决“是不是这版状态”的并发问题。审批页面重复点击、旧页面回放、过期恢复请求，这些 stale resume 场景都需要 token 和 version 双重校验，单独校验任何一个都不够。

项目落地

当前恢复前会先取 Java 里的 checkpoint，再比对 `resumeToken + checkpointVersion`；任一不匹配，都会拒绝恢复。

易错表达

- 易错：只说“多做一层校验更安全”
- 更稳：直接讲“它解决 stale page 和陈旧状态恢复问题”

大厂式继续追问

为什么只校验 token 还不够？

面试题 13：现在的 replay / continue 和审批恢复分别怎么讲？

标准回答

审批恢复仍然是最标准的 HITL 恢复路径，通过 `resumeToken + checkpointVersion` 做恢复校验；在此之外，当前项目已经支持从最新 checkpoint 做 `replay / continue`，所以恢复能力不再局限于审批态。更稳的说法是：项目已经具备 Java-backed checkpoint store 和最新 checkpoint continue 能力，但不会把它说成官方 durable checkpointer 的完全等价实现。

项目落地

README 和 Demo 文档都已经写明：当前支持从最新 checkpoint 做 replay / continue；审批恢复依然保留 `resumeToken + checkpointVersion` 路径。

易错表达

- 易错：还沿用“现在主要只能审批恢复”的旧说法

- 更稳：区分两条路径，“审批恢复最完整，非审批态也能做最新 checkpoint continue”

大厂式继续追问

如果面试官继续问“那是不是任意 crash 都能自动恢复到精确节点继续执行”，你怎么守住边界？

面试题 14：human approval 为什么是 runtime 的正式分支？

标准回答

因为审批不是页面装饰，而是系统风控边界的一部分。高风险 SQL、reviewer 低置信度、证据冲突这些场景，都不应该让模型自己继续放大风险，而应该进入 `human_approval` 节点，由人做最终确认或拒绝。

项目落地

当前 runtime 有显式 `human_approval` 节点，支持 `MANUAL / AUTO_APPROVE / AUTO_REJECT` 策略；Java 侧有 `agent_human_task` 和配套恢复接口，Eval 也能覆盖审批分支。

易错表达

- 易错：把审批讲成“为了演示好看加的交互”
- 更稳：讲“这是风控、可审计和恢复路径的一部分”

大厂式继续追问

如果审批被拒绝，为什么系统不能绕过高风险工具继续执行？

模块五：tool plane / MCP / governance

面试题 15：MCP 在这个项目里到底是什么角色？

标准回答

MCP 在这里不是面试名词，而是工具平面的统一接入方式。更稳的讲法是：Java control plane 暴露 MCP 风格的 tool plane，Python runtime 通过工具调用接入它，这样工具能力、工具治理和系统记录源都收敛在 Java 一侧，而不是散落在 runtime 各处。

项目落地

当前工具链至少包括 `kb_search`、`doc_fetch`、`sql_query`，并附带 `riskLevel / approvalPolicy / timeoutBudget / retryPolicy / auditRequired` 这些治理元数据。

易错表达

- 易错：把 MCP 讲成“我支持一个流行协议”
- 更稳：讲“它是工具接入与治理边界的一部分”

大厂式继续追问

如果面试官说“为什么不让 runtime 直接调业务服务”，你怎么回答？

面试题 16：为什么工具平面要保留在 Java control plane，而不是让 Python 直接调业务服务？

标准回答

因为工具不仅是功能调用点，还是治理对象。把工具收敛在 Java control plane 里，权限、审计、风控元数据、运行记录和后续扩展都能统一管理；如果直接让 Python 调各个业务服务，工具边界会散，记录源也会变得不完整。

项目落地

Python runtime 当前通过 LangChain tools 调 Java MCP tool plane，而 Java 再统一承接业务工具与运行态落库。

易错表达

- 易错：只说“更解耦”
- 更稳：讲“权限收口、审计一致、工具治理、可扩展性”这四个工程收益

大厂式继续追问

为什么这里不是“多包了一层壳”，而是“把工具平面固定下来”？

面试题 17：`kb_search` / `doc_fetch` / `sql_query` 三个工具为什么要同时存在？

标准回答

因为它们解决的不是同一种信息需求。`kb_search` 负责先做知识候选召回，`doc_fetch` 负责把具体文档或 chunk 内容进一步拿全，`sql_query` 负责结构化数据查询。把三者分开后，知识证据、文档证据和数据证据就可以在 runtime 里被显式编排和审查。

项目落地

当前 runtime 会根据 route 和 pending tasks 选择不同工具路径，reviewer 再结合结果判断 grounded、replan 或 approval。

易错表达

- 易错：把三个工具讲成“只是多准备了几个能力点”
- 更稳：强调它们分别对应“召回、补全内容、结构化数据”三类证据来源

大厂式继续追问

如果面试官说“有了 `kb_search` 为什么还要 `doc_fetch`”，你怎么答？

面试题 18：这个项目里的工具治理和 SQL 风控怎么讲更像做过真实系统的人？

标准回答

更稳的讲法不是“我限制了 SQL”，而是“我把工具当成有风险等级和审批策略的治理对象”。企业场景里，风险不只来自模型，还来自它能调用什么工具、调用到什么范围、失败后怎么重试、是否需要审计。SQL 只是最容易被追问的一类高风险工具。

项目落地

当前工具元数据里已经有 `riskLevel / approvalPolicy / timeoutBudget / retryPolicy / auditRequired`；高风险 SQL 会触发人工审批，审批决策还会进入 checkpoint、human task 和 Eval 记录。

易错表达

- 易错：把风控讲成“SQL 只读就没事了”
- 更稳：讲“工具风险是系统级问题，SQL 只是其中最典型的一类”

大厂式继续追问

为什么 `approvalPolicy`、`auditRequired` 这些字段比单纯“只读 SQL”更有工程味？

模块六：event stream / trace / observability

面试题 19：`agent_run_event + SSE event stream` 为什么是当前版本的重要增强？

标准回答

因为 Agent 系统不是只需要最终答案，还需要知道运行过程中发生了什么。事件流把“开始执行、节点切换、工具调用、进入审批、恢复、完成”等过程显式暴露出来，前端能实时看到 run 的变化，trace 也更容易解释。

项目落地

Java control plane 维护 `agent_run_event`，前端通过 SSE 订阅 live events，轮询继续保留为兼容回退，这已经写进当前 README 和 Demo 路径。

易错表达

- 易错：还沿用“前端主要靠轮询”的旧口径
- 更稳：明确说“当前主讲 SSE 实时事件流，轮询只是兼容兜底”

大厂式继续追问

为什么事件流对 Agent 项目更重要，而不只是工作台体验优化？

面试题 20：为什么 step trace、graph trace 和 nodePath 对这个项目是硬需求？

标准回答

因为 Agent 项目不能只看最终答案，还要看“它是不是按合理路径得到这个答案”。step trace 解决执行过程可追踪，graph trace 解决节点流转可解释，nodePath 解决 Eval 和线上路径是否一致。这些信息决定了你能不能定位问题、复现问题和评估质量。

项目落地

当前系统会沉淀 run/step/event/checkpoint/eval，前端还能直接看 graph 和 trace，Eval 里也记录 `nodePath` 等运行信息。

易错表达

- 易错：把 trace 讲成“方便页面展示”

- 更稳：讲“它是调试、评测、追责和演示的共同基础设施”

大厂式继续追问

为什么 nodePath 在 Agent 项目里比传统接口日志更重要？

模块七：runtime-based Eval

面试题 21：为什么你现在说 Eval 更“真”了？

标准回答

因为现在 Eval 不再走旁路脚本或旧逻辑，而是直接创建真实 run 去执行 runtime 图。这样评测路径和线上执行路径是一致的，评测结果才能真正反映系统主链的质量，而不是只评一个被简化的离线流程。

项目落地

当前 README、讲稿和代码里都已经明确：Eval 直接跑 runtime 图，并记录 `route / grounding / citation / approval / latency / retry / orchestrationMode / skills` 等维度。

易错表达

- 易错：把 Eval 讲成“离线随便跑几条 case 看看”
- 更稳：讲“它跑的是真实主链，所以和线上行为一致”

大厂式继续追问

如果面试官问“为什么这点比有个 dashboard 更重要”，你怎么答？

面试题 22：为什么 Eval 要记录 `route / grounding / citation / approval / latency / retry / orchestrationMode / skills` 这些维度？

标准回答

因为 Agent 质量不是一个单分数能说明白的。你不仅要知道最后答得对不对，还要知道路由是否合理、证据是否扎实、引用是否到位、是否触发审批、延迟和重试是否失控、用了哪种编排模式、技能调用是否合适。把这些维度拆开后，Eval 才能真正指导优化。

项目落地

当前项目的 Eval 闭环已经把这些维度写进 README 和 JD 映射矩阵，前端还有对应 dashboard 展示。

易错表达

- 易错：把 Eval 讲成“就是一个 accuracy 分数”
- 更稳：明确说“Agent 质量是多维度的，路径和代价同样重要”

大厂式继续追问

为什么 `orchestrationMode` 和 `skills` 也值得进入 Eval，而不是只看答案文本？

模块八：可靠性、边界与诚实口径

面试题 23：这个项目对无 key、结构化输出失败和 runtime 异常分别怎么收口？

标准回答

这三类问题要分开讲。没有 `GEMINI_API_KEY` 时，系统会自动退回 mock 模式，保证本地开发和 CI 稳定可跑；结构化输出失败时，runtime 会有限次重试，超限后进入 bounded failure 或 mock fallback；runtime 异常时，最后一次已提交状态不会只存在 Python 内存里，因为 checkpoint 已经回到 Java control plane，并且当前支持从最新 checkpoint 做 replay / continue。

项目落地

当前 README 已明确说明 live / mock 两种模式；runtime 里已经有 Java-backed checkpoint 和 replay 路径。

易错表达

- 易错：回答成“异常了就从头再跑一遍”
- 更稳：区分“无 key”“结构化输出失败”“运行态恢复”三类不同收口路径

大厂式继续追问

为什么要保留 mock 模式，而不是所有环境都强依赖真实模型？

面试题 24：现在这个项目离更完整的生产级平台还差什么？

标准回答

更稳的回答不是装成“已经是完整平台”，而是明确说主骨架已经到位，但还差几类增强：更强的 prompt / schema 治理、更完整的权限与多租户能力、更接近官方 durable checkpointer 的兼容能力，以及规模化运维和平台化治理。当前已经具备的是系统主链和关键运行态骨架，不是最终 SaaS 平台。

项目落地

README 已经明确诚实边界：不是官方 durable checkpointer，不是完整 SaaS 权限平台，也没有把系统做成完全开放式通用 multi-agent 平台。

易错表达

- 易错：为了显得高级，硬说“已经是完整 Agent 平台”
- 更稳：讲“主骨架到位，当前边界清晰，后续是平台增强而不是推翻重写”

大厂式继续追问

为什么明确边界，反而会让面试官更信你做过项目？

模块九：与 AISmartQA 的区分

面试题 25：`AgentOps` 和 `AISmartQA` 为什么不重复？

标准回答

两者都属于 AI 项目，但解决的问题层次不同。`AISmartQA` 更偏工程化 RAG 和知识库问答后端，重点在文档接入、异步处理、检索设计、事务边界和后端治理；`AgentOps` 更偏 Agent runtime 与系统治理，重点在状态化

执行、checkpoint、approval、event stream、tool governance 和 runtime-based Eval。前者解决“知识怎么接进来并检索得更稳”，后者解决“多步执行怎么被治理、恢复和评测”。

项目落地

`AISmartQA` 的主链是 `文档上传 -> 异步解析 -> embedding -> ES + Milvus -> Hybrid + rerank -> Gemini 问答`；`AgentOps` 的主链是 `run -> runtime graph -> tool -> review / approval -> checkpoint / event / eval`。

易错表达

- 易错：回答成“一个偏后端，一个偏 AI”
- 更稳：明确区分“知识接入与检索增强” vs “运行时编排与系统治理”

大厂式继续追问

如果面试官说“你这两个项目看起来都能回答问题”，你怎么把差异讲深一层？

面试题 26：为什么 `AgentOps` 应该放在 AI 应用 / Agent 岗简历的第一主项目？

标准回答

因为它更直接命中这类岗位最爱问的关键词和系统能力：`LangGraph runtime`、`checkpoint replay`、`human-in-the-loop`、`tool governance`、`event stream`、`runtime Eval`、`orchestration mode`。`AISmartQA` 很强，但它更像工程化 RAG 后端；`AgentOps` 则更自然地映射到 Agent 系统、运行时和控制面这些岗位核心语境。

项目落地

当前 `AgentOps` 已经形成 `Java control plane + Python runtime + React workspace` 的完整三层系统，面试时不只可以讲主链，还可以讲 graph、审批、事件流和 Eval dashboard 的现场演示证据。

易错表达

- 易错：排序理由只说“这个项目更新”
- 更稳：讲“它更贴合岗位语境，能稳定承接 LangGraph / runtime / HITL / Eval 这类深挖”

大厂式继续追问

如果面试官只给你一个主项目展开，你为什么会选 `AgentOps`？